

To preface this, I try to avoid AI for school work as I believe I need to improve my personal skill first before incorporating AI into my workflow.

My prompt was mainly from the assignment list with some modification and I removed the unnecessary parts as seen below

Write a C or C++ program that reads a line of text and extracts a single valid IPv4 address — optionally followed by a port number — embedded anywhere in that text. Only digits, periods (.), and colons (:) are ever part of a valid token; every other character is garbage and is skipped. A candidate token must match the address grammar in full — no partial matches, no truncating to find a valid piece inside a longer run.

An address is four octets separated by periods (octet.octet.octet.octet), each octet 1–3 digits, value 0–255, no leading zero unless the value is exactly 0. An optional **:port** may follow the fourth octet: 1–5 digits, value 0–65535, same leading-zero rule. If a colon is present, the port must be fully valid or the entire match — address included — is rejected.

Additional rules:

Built-in functions/libraries you may not use

- Any string-to-number conversion function: `atoi`, `atol`, `atoll`, `strtol`, `strtoul`, `strtod`, `stoi`, `stol`, `stoul`, `sscanf`, `scanf` with numeric conversions.
- Any address-parsing library function: `inet_aton`, `inet_pton`, `inet_addr`, or equivalents.
- Any regular-expression facility (`std::regex`, `POSIX regex.h`, or similar) — the parsing and validation logic must be your own character-by-character code, not a pattern matched by a library.
- Standard character-classification functions (`isdigit`, etc.) are fine to use.

Other requirements

- Exactly one valid address may be extracted per input line; everything else in the line is either garbage (skipped) or part of a candidate token that fails validation.
- Reject anything that does not exactly match the grammar above — wrong octet count, empty octet, out-of-range octet or port, a disallowed leading zero, a second colon, a colon not immediately after the fourth octet, or a stray period/colon directly adjacent to an otherwise-valid address.
- On success, print: **Extracted IPv4 address: A.B.C.D (decimal value: N, port: P)** where N is the 32-bit decimal value and P is the port number or the literal text `none`.

Program requirements

- Input loop: `main` continuously prompts the user for input until the user enters END (case-sensitive), then prints Program terminated. and exits.
- Extraction function: implement `extractIPv4` exactly as prototyped above. All digit accumulation must be done by hand.
- Display: `main` receives the result from `extractIPv4` and formats the output exactly as specified above.

The sample run should look as below

```
Enter a string (or 'END' to quit): connecting to 192.168.1.1 now
Extracted IPv4 address: 192.168.1.1 (decimal value: 3232235777, port:
none)
Enter a string (or 'END' to quit): server=10.0.0.255:8080end
Extracted IPv4 address: 10.0.0.255 (decimal value: 167772415, port:
8080)
Enter a string (or 'END' to quit): 192a168.1.1.1
Extracted IPv4 address: 168.1.1.1 (decimal value: 2818638081, port:
none)
Enter a string (or 'END' to quit): 192.168.1.1.
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): Connection from 192.168.1.1 refused
Extracted IPv4 address: 192.168.1.1 (decimal value: 3232235777, port:
none)
Enter a string (or 'END' to quit): 192.168.01.1
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): 1.2.3.4:99999
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): 12.34.56
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): no number here
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): END
Program terminated.
```

AI responded with a file as seen in the Github and a hefty description of the code

## Approach

`extractIPv4` splits the problem into three layers:

1. **Run detection.** The function scans the line and skips every character that isn't a digit, `.`, or `:`. When it hits one of those characters, it collects the maximal contiguous run of them.
2. **Whole-run validation.** Each run is validated in full against `octet.octet.octet.octet[:port]`. A run that fails is discarded entirely. The code never retries a sub-range of a failed run, which is what enforces the "no partial matches / no truncating" rule. This is why `192.168.1.1.` and `.192.168.1.1` are rejected, while `192a168.1.1.1` yields `168.1.1.1` (the letter splits the input into two separate runs).
3. **Field parsing.** A shared helper handles both octets (max 3 digits, max 255) and ports (max 5 digits, max 65535). It accumulates digits by hand (`val = val * 10 + (c - '0')`), rejects a leading zero unless the field is exactly one digit `0`, and rejects a digit run longer than the allowed width instead of truncating it.

## Design decisions

- If a colon is present, the port must be fully valid or the whole run is rejected. There is no fallback to the bare address, so `1.2.3.4:99999` is invalid.
- Leftover characters after the port (for example a second colon) fail the run, because the run must be consumed exactly.
- The 32-bit value is built with shifts: `(o1<<24)|(o2<<16)|(o3<<8)|o4`. `main` recovers A.B.C.D from that value with shifts and masks.
- Only `isdigit` is used from the standard library. No conversion functions, address-parsing functions, or regex.
- `main` uses `getline` so that lines containing spaces work, and it exits only on an exact, case-sensitive `END`.

Now, I tested with the code and this might be my fault but I had a hard time finding any bugs that the code has output. To reiterate, the code in the Github is solely written by Claude with little to none modification from me.